

Application de salon de discussion en temps réel

ASMA KHOULA
MOHAMED AMINE JILANI
Génie Informatique

Encadrant
Mohammad Beyrouti

UTC – SR03
Année universitaire 2025–2026

Table des matières

1 Introduction **4**

1.1 Contexte du projet 4

1.2 Objectifs 4

1.3 Organisation du rapport 5

2 Architecture de l'application **6**

2.1 Vue d'ensemble 6

2.2 Arborescence du projet 7

2.2.1 Côté Spring Boot 7

2.2.2 Côté React 9

3 Conception **10**

3.1 Diagramme de classes 10

3.1.1 Description des entités 10

3.1.2 Schéma relationnel 12

3.2 Diagramme de cas d'utilisation 12

3.2.1 Cas d'utilisation de l'utilisateur standard 12

3.2.2 Cas d'utilisation de l'administrateur 13

4 Interface d'administrateur (MVC) **15**

4.1 Architecture générale 15

4.2 Organisation des contrôleurs 16

4.3 Gestion de la session et mécanisme de sécurité 17

4.3.1 Mécanisme de blocage après 3 tentatives échouées 17

4.3.2 Protection des routes 18

4.4 Gestion des utilisateurs 18

5 Interface Utilisateur (React + REST) **19**

5.1 Principe général : deux applications indépendantes 19

5.2 Absence d'état partagé entre React et Spring 20

5.3 Organisation des services REST côté Spring 20
5.4 Organisation du code côté React 21
5.4.1 Cycle de vie d'une page React 21
5.5 Fonctionnalités développées 22

6 Service de chat WebSocket **23**
6.1 Pourquoi WebSocket plutôt que HTTP classique 23
6.2 Le choix de STOMP 23
6.3 Fonctionnement général 23
6.3.1 Non-persistance des messages 24
6.3.2 Liste des utilisateurs connectés et limite connue 24

7 Limites et pistes d'améliorations **25**
7.1 Déploiement de l'application React non intégré à Spring 25
7.2 Autres améliorations fonctionnelles envisageables 25
7.2.1 Gestion des déconnexions WebSocket 26
7.2.2 Envoi réel des identifiants par courrier électronique 26

8 Aspects complémentaires **27**
8.1 Organisation du travail en équipe 27
8.2 Utilisation de l'IA 27

9 Conclusion **29**

Chapitre 1

Introduction

1.1 Contexte du projet

Le projet a comme finalité le développement d'une application de salon de discussion (chat). Dans cette application, chaque utilisateur peut programmer un canal de discussion à une date donnée, avec une durée de validité déterminée, et inviter au moins un autre utilisateur à le rejoindre. La particularité principale de l'application est qu'elle ne sauvegarde pas le contenu des messages échangés (ni leur contenu, ni l'heure d'envoi). Les messages sont uniquement diffusés en temps réel sur les sessions des utilisateurs connectés à un même canal. Une conséquence directe de ce choix est qu'un utilisateur qui se connecte tardivement à un canal ne pourra pas consulter les messages publiés avant son arrivée. Un canal de discussion est caractérisé par un identifiant unique, un titre, une description, une date avec un horaire de début, et une durée de validité. La gestion des utilisateurs (création, désactivation, suppression) est centralisée et accessible uniquement via une interface d'administration réservée aux comptes de type administrateur.

1.2 Objectifs

Le projet est structuré en deux parties distinctes, correspondant à deux architectures différentes. La première partie concerne le développement de l'application web pour les fonctionnalités de base : gestion des utilisateurs, planification, édition et affichage de la liste des salons de discussion. Elle se compose de deux interfaces : une interface d'administration suivant l'architecture MVC (modèle en JPA/ORM, vues en Thymeleaf, contrôleurs Spring), et une interface utilisateur suivant une architecture single page (composants React consommant des API REST). Les deux interfaces exploitent le même modèle métier et donc la même couche d'accès aux données. La seconde partie concerne le développement du serveur de chat à proprement parler, utilisant le protocole WebSocket

via l'API Java WebSocket (ou son équivalent Spring). Ce serveur reçoit les messages envoyés depuis la fenêtre de chat et les diffuse en temps réel sur le canal concerné, auprès de tous les clients qui y sont connectés.

1.3 Organisation du rapport

Ce rapport présente l'architecture globale de l'application avec les schémas correspondants, la conception détaillée (diagramme de classes, schéma relationnel, diagramme de cas d'utilisation) avec la justification des choix effectués, puis les explications sur les interactions entre les différentes technologies utilisées : React, Spring et WebSocket. Une synthèse des mesures de sécurité ainsi qu'un bilan des limites et améliorations possibles complètent ce document avant la conclusion.

Chapitre 2

Architecture de l'application

2.1 Vue d'ensemble

L'application ChattyChat repose sur une seule application Spring Boot, qui expose simultanément trois façades technologiques différentes selon le type de client qui s'y connecte. Un administrateur, depuis son navigateur, interagit avec des pages HTML générées côté serveur via Thymeleaf. Un utilisateur standard, depuis l'application React, communique avec le serveur exclusivement via une API REST échangeant des données au format JSON. Enfin, dès qu'un utilisateur (administrateur ou non) ouvre une fenêtre de discussion, une troisième voie de communication s'active : une connexion WebSocket dédiée à la diffusion des messages en temps réel.

Ces trois façades ne sont pas indépendantes : elles s'appuient toutes sur le même socle de données, à savoir les mêmes entités métier (User, Chat, Invitation) et les mêmes repositories Spring Data JPA, connectés à une unique base de données MySQL. C'est cette mutualisation qui permet de respecter l'exigence du cahier des charges selon laquelle l'interface utilisateur doit exploiter « le même modèle et donc la même couche d'accès aux données » que l'interface d'administration.

Le schéma suivant illustre cette organisation générale.

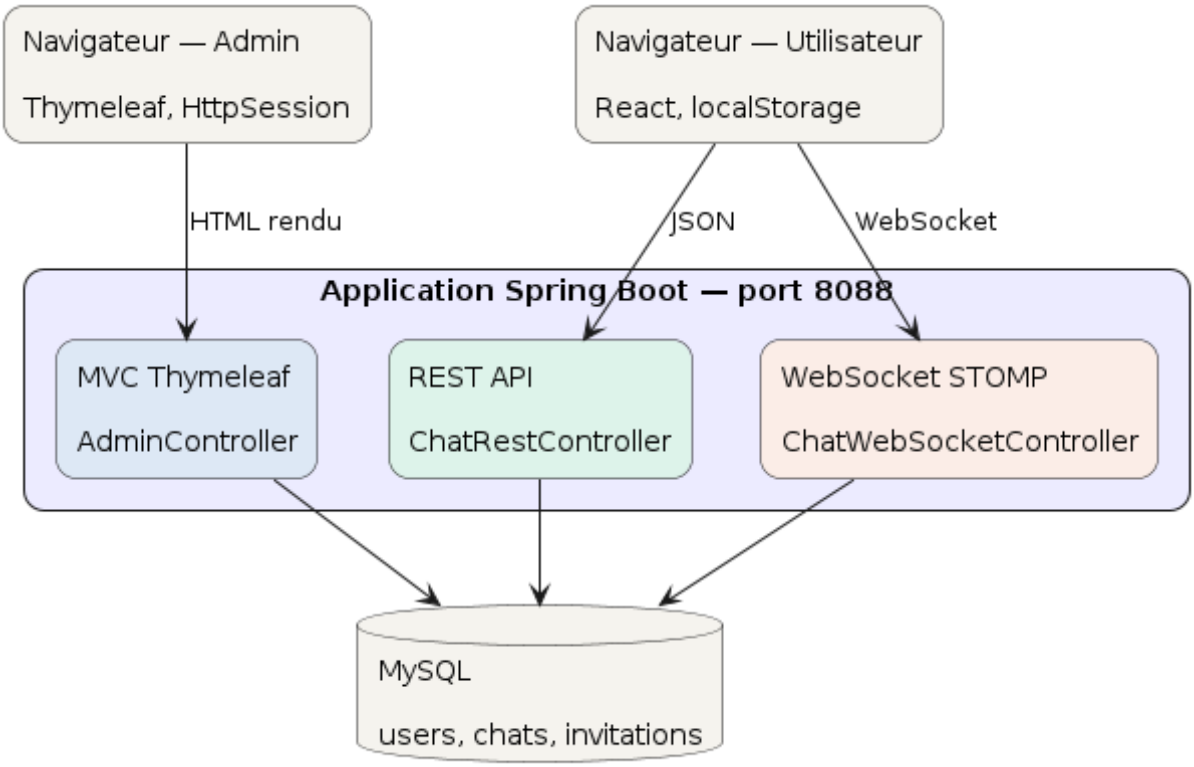


FIGURE 2.1 – Vue d'ensemble : les deux navigateurs, l'application Spring Boot unique avec ses trois façades MVC / REST / WebSocket, et la base MySQL partagée

2.2 Arborescence du projet

Le projet est organisé en deux dépôts distincts, correspondant à la séparation technique déjà évoquée : un projet Spring Boot (Maven) regroupant l'interface d'administration, l'API REST et le serveur WebSocket, et un projet React (Vite) regroupant l'interface utilisateur. Cette organisation reflète directement le fait que les deux applications sont déployées et exécutées indépendamment l'une de l'autre, comme expliqué en partie 5.1.

2.2.1 Côté Spring Boot

`src/main/java/chattychat/sr03_chat_app/`

```
model/
   User.java              → entité utilisateur (nom, email, mot de passe, rôle...)
   Chat.java              → entité salon de discussion
   Invitation.java        → entité de liaison User/Chat avec statut
   StatutInvitation.java  → énumération des statuts d'invitation
```

```
repository/
   UserRepository.java       → accès aux données des utilisateurs (JPA)
   ChatRepository.java       → accès aux données des chats (JPA)
   InvitationRepository.java → accès aux données des invitations (JPA)

controller/
   AdminAuthController.java  → connexion / déconnexion de l'administrateur
   AdminController.java      → gestion des utilisateurs (CRUD, pagination)
   ChatController.java       → gestion des chats et des invitations
   AuthController.java       → connexion / déconnexion des utilisateurs standards

rest/
   AuthRestController.java   → authentification pour le client React
   UserRestController.java   → informations utilisateur en JSON
   ChatRestController.java   → CRUD des chats et invitations en JSON

dto/   [objets de transfert utilisés par l'API REST]
   UserDTO.java
   ChatDTO.java
   ChatRequestDTO.java
   LoginRequest.java
   InviteRequestDTO.java

websocket/   [serveur de chat temps réel]
   WebSocketConfig.java          → configuration du endpoint STOMP et des topics
   ChatWebSocketController.java  → réception et diffusion des messages
   ChatMessage.java              → objet de transport (non persisté)

config/
   SecurityConfig.java   → désactivation du filtrage Spring Security par défaut,
             bean BCryptPasswordEncoder pour le hachage des mots de passe

Sr03ChatAppApplication.java   → point d'entrée de l'application Spring Boot

src/main/resources/
templates/admin/    → vues Thymeleaf de l'interface d'administration
```

```
templates/auth/     → vues Thymeleaf de connexion utilisateur standard
static/css/         → feuille de style partagée par toutes les pages Thymeleaf
static/js/          → scripts JavaScript (validation, filtres, mot de passe)
```

2.2.2 Côté React

```
src/
 pages/      [une vue complète de l'application par fichier]
    Login.jsx            → formulaire de connexion utilisateur
    Logout.jsx           → déconnexion (vidage du localStorage)
    Home.jsx              → accueil, liste des chats (propriétaire + invité)
    PlanifierChat.jsx     → formulaire de création d'un chat + sélection des invités
    MesSalons.jsx         → gestion des chats dont l'utilisateur est propriétaire
    MesInvitations.jsx    → liste des chats où l'utilisateur est invité
    ChatRoom.jsx          → fenêtre de discussion en temps réel (WebSocket)

 components/
    ProtectedRoute.jsx → protège l'accès aux pages réservées aux utilisateurs connecté

 services/
    api.js    → centralise l'ensemble des appels à l'API REST (Axios)

 App.jsx       → définition des routes et structure générale de l'application
 App.css       → styles partagés par l'ensemble des composants React
```

Chapitre 3

Conception

3.1 Diagramme de classes

La modélisation des données métier de l'application repose sur trois entités principales User, Chat et Invitation ainsi qu'une énumération StatutInvitation. Ce modèle a été conçu pour répondre précisément aux quatre besoins fonctionnels identifiés dans le cahier des charges : la gestion des utilisateurs, la gestion des chats, la récupération de la liste des chats dont un utilisateur est propriétaire, et la récupération de la liste des chats auxquels il est invité.

3.1.1 Description des entités

L'entité User représente un utilisateur de la plateforme, qu'il soit administrateur ou utilisateur standard. Elle porte les attributs identitaires (nom, prénom, email unique), un mot de passe haché, ainsi que plusieurs champs de gestion d'état : un booléen admin distinguant les deux profils, un booléen actif permettant de désactiver un compte sans le supprimer définitivement, un booléen bloque et un compteur nbTentatives mis en œuvre pour la sécurité du mécanisme de connexion.

L'entité Chat représente un salon de discussion. Outre son titre et sa description, elle porte une date de début (dateCreation) et une durée de validité en minutes (dureeValiditeMinutes). Deux méthodes métier ont été ajoutées directement sur cette entité : getDateExpiration(), qui calcule la date de fin de validité en additionnant la durée à la date de début, et isValable(), qui détermine si le chat est encore actif en comparant cette date d'expiration à l'instant présent. Le choix de placer cette logique directement dans l'entité plutôt que dans une couche de service séparée se justifie par sa simplicité et son caractère intrinsèquement lié à l'état de l'objet Chat lui-même.

L'entité Invitation fait le lien entre un User et un Chat, en y associant un statut

représenté par l'énumération StatutInvitation (EN ATTENTE, ACCEPTEE, REFUSEE).

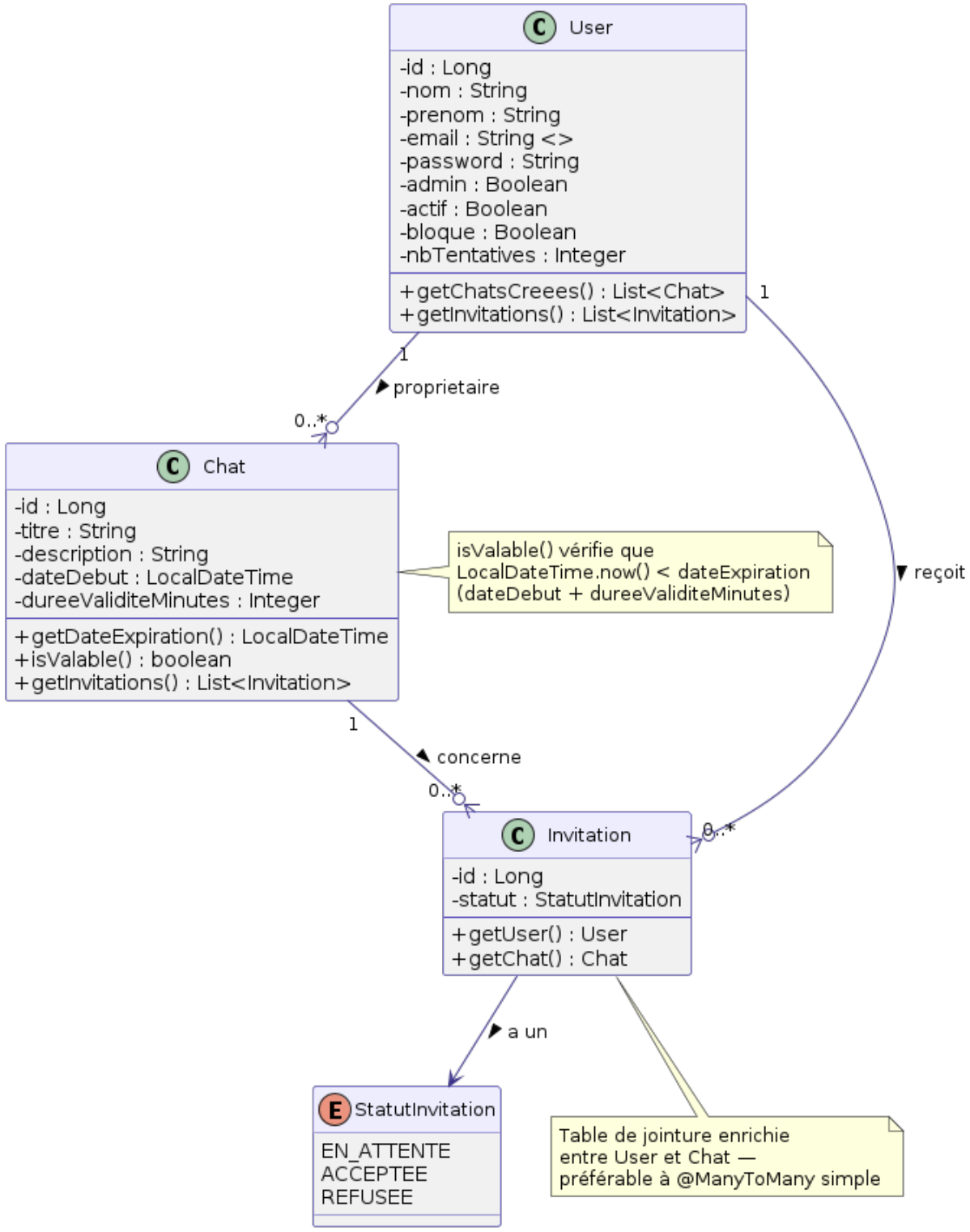


FIGURE 3.1 – Diagramme de classes de l'application ChattyChat

3.1.2 Schéma relationnel

Le schéma relationnel découle directement du diagramme de classes présenté précédemment, conformément au mapping objet-relationnel réalisé par Hibernate via les annotations JPA des entités `User`, `Chat` et `Invitation`.

```
users (user_id PK, nom, prenom, email UNIQUE, password,
       admin, actif, bloque, nb_tentatives)

chats (chat_id PK, titre, description, date_creation,
       duree_validite_minutes, proprietaire_id FK -> users.user_id)

invitations (invitation_id PK, user_id FK -> users.user_id,
             chat_id FK -> chats.chat_id, statut)
```

Trois remarques accompagnent ce schéma.

D'une part, la contrainte d'unicité sur le champ `email` de la table `users` garantit qu'aucun doublon de compte ne peut exister, condition nécessaire au bon fonctionnement de l'authentification par adresse email.

D'autre part, la clé étrangère `proprietaire id` dans la table `chats` matérialise la relation `@ManyToOne` entre un chat et son propriétaire : un utilisateur peut posséder plusieurs chats, mais un chat n'a qu'un seul propriétaire.

Enfin, la table `invitations` porte deux clés étrangères distinctes vers `users` et `chats`, ce qui en fait une table de jointure enrichie. Elle inclut également le champ `statut`, permettant de suivre l'état de chaque invitation indépendamment des deux autres tables.

3.2 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation permet de visualiser l'ensemble des interactions possibles entre les acteurs du système et les fonctionnalités de l'application.

3.2.1 Cas d'utilisation de l'utilisateur standard

L'utilisateur interagit exclusivement avec l'interface React. Le cas d'utilisation « Planifier un chat » inclut systématiquement « Inviter des utilisateurs », puisque la sélection des invités fait partie intégrante du formulaire de création, conformément au choix d'implémentation décrit en partie 5.5. L'ouverture d'une fenêtre de chat constitue une extension des deux cas de consultation (chats propriétaires et invitations), ces deux pages constituant les deux points d'entrée possibles vers la fenêtre de discussion. Ce dernier

cas d'utilisation dépend à son tour du service de chat WebSocket, représenté comme un second système englobant les cas « Envoyer un message » et « Voir les connectés ».

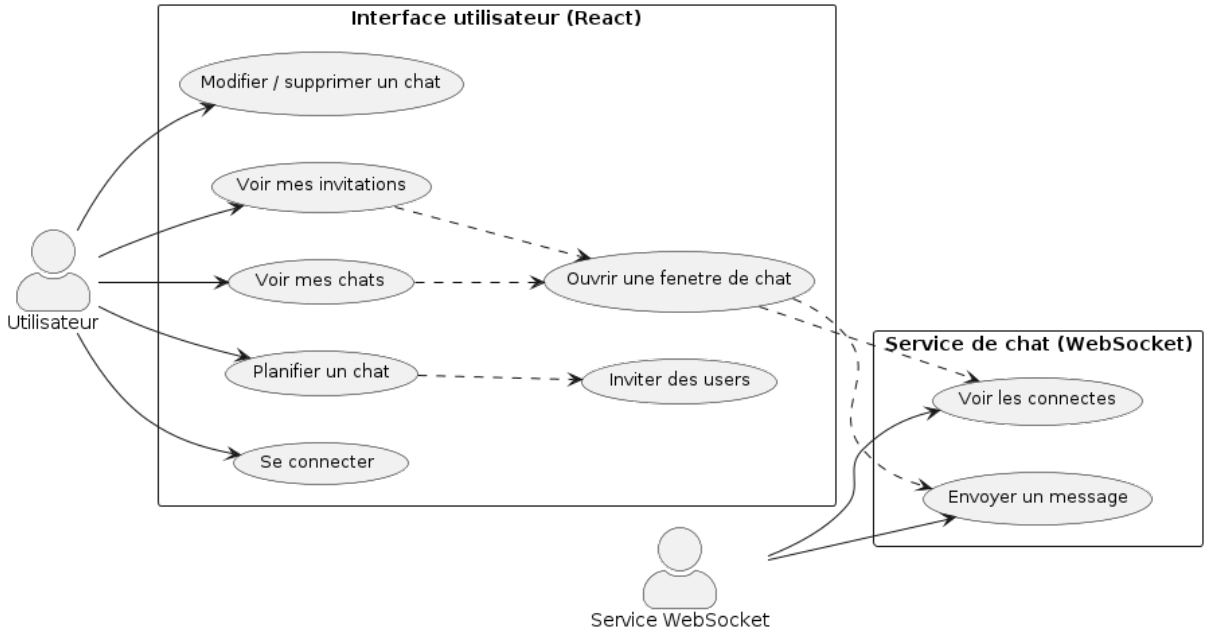


FIGURE 3.2 – Cas d'utilisation Utilisateur

3.2.2 Cas d'utilisation de l'administrateur

L'administrateur interagit exclusivement avec l'interface MVC. Ses cas d'utilisation se répartissent naturellement en deux groupes fonctionnels distincts, représentés par deux conteneurs séparés sur le diagramme : la gestion des comptes utilisateurs d'une part, la gestion des salons de discussion d'autre part. Le cas « Gérer les invitations » constitue une extension du cas « Créer ou modifier un chat », puisque l'administrateur peut y accéder soit immédiatement après la création d'un chat, soit ultérieurement depuis la liste des salons existants.

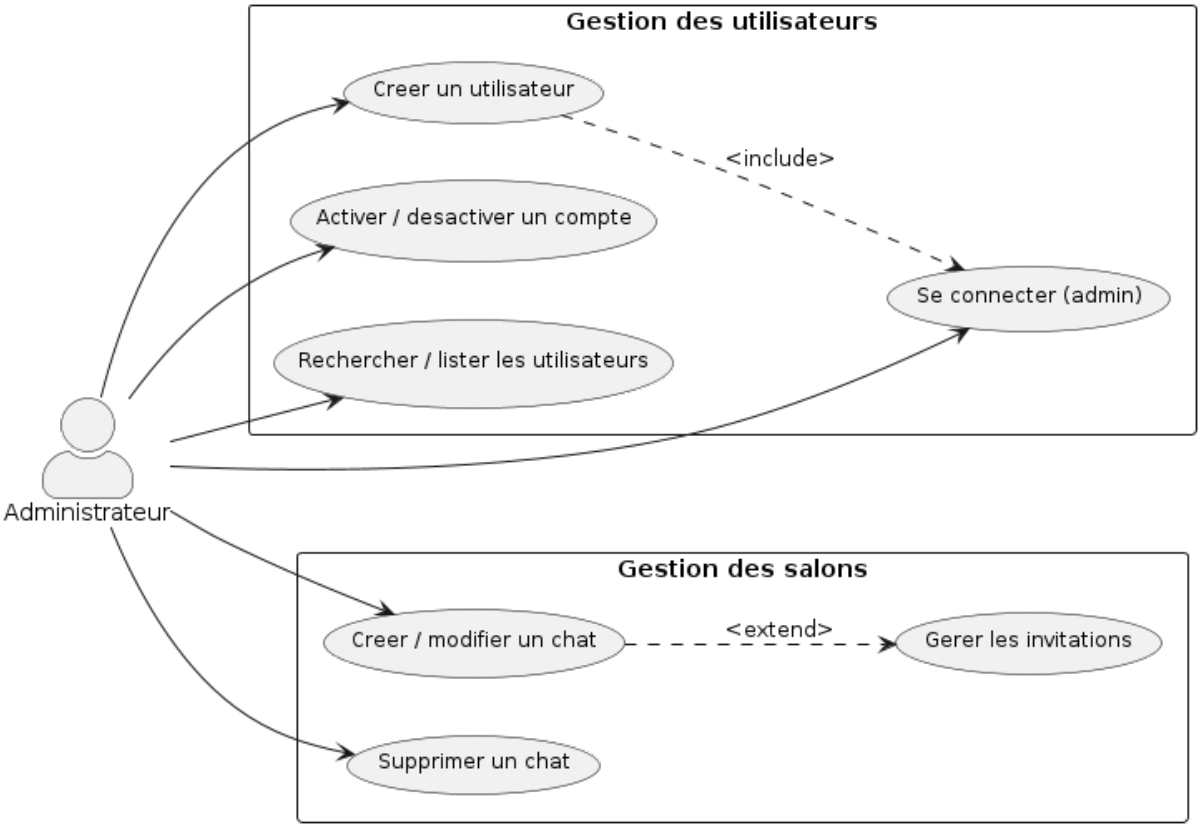


FIGURE 3.3 – Cas d'utilisation Administrateur

Chapitre 4

Interface d'administrateur (MVC)

4.1 Architecture générale

L'interface d'administration suit le patron de conception **Modèle-Vue-Contrôleur**, dans son implémentation classique fournie par Spring MVC. Cette architecture organise le code en trois couches aux responsabilités strictement séparées, ce qui facilite à la fois la lisibilité du code et son évolution.

Le modèle correspond aux entités métier présentées dans la partie conception (User, Chat, Invitation), enrichies des annotations JPA permettant leur persistance, ainsi qu'à la couche d'accès aux données constituée par les interfaces repository. Le contrôleur correspond aux classes annotées `@Controller`, qui interceptent les requêtes HTTP entrantes et orchestrent le traitement à effectuer. La vue correspond aux templates Thymeleaf, responsables de la génération du HTML final renvoyé au navigateur.

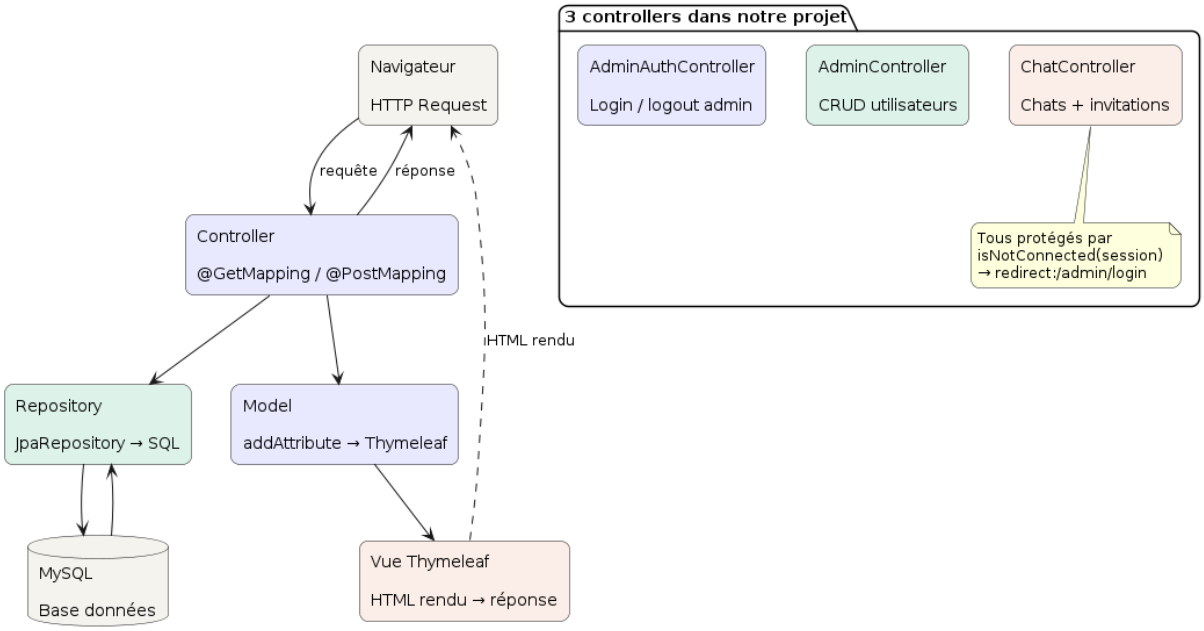


FIGURE 4.1 – Schéma du flux d'une requête admin

Le déroulement d'une requête type illustre cette organisation. Lorsqu'un administrateur clique sur un lien ou soumet un formulaire, le navigateur envoie une requête HTTP interceptée par la méthode du contrôleur correspondant, grâce aux annotations `@GetMapping` ou `@PostMapping`.

Ce contrôleur délègue l'accès aux données à un repository, qui interroge la base MySQL via Hibernate sans qu'aucune requête SQL n'ait été écrite manuellement. Spring Data JPA génère automatiquement ces requêtes à partir du nom des méthodes.

Les données récupérées sont ensuite placées dans l'objet `Model` via `addAttribute()`, avant que le contrôleur ne retourne le nom logique d'une vue. Thymeleaf interprète ensuite les attributs `th:*` dans le template HTML afin de produire la page finale envoyée au navigateur.

4.2 Organisation des contrôleurs

Le projet comporte quatre contrôleurs spécialisés, chacun responsable d'un domaine fonctionnel distinct.

- **AdminAuthController** : gestion de l'authentification administrateur (connexion, déconnexion).
- **AdminController** : gestion CRUD des utilisateurs (pagination, recherche, tri).
- **ChatController** : gestion des salons de discussion et des invitations.
- **AuthController** : authentification des utilisateurs standards (créés uniquement

par l'administrateur).

4.3 Gestion de la session et mécanisme de sécurité

La sécurité de l'interface d'administration repose sur le mécanisme de session HTTP fourni par Spring, sans utiliser Spring Security pour l'authentification.

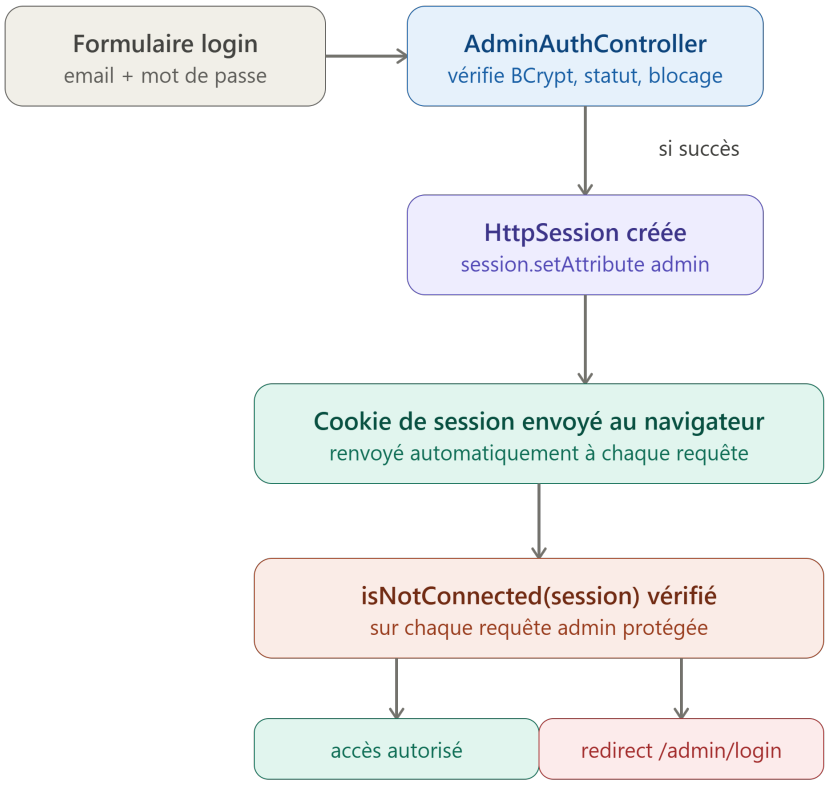


FIGURE 4.2 – Schéma du mécanisme HttpSession

4.3.1 Mécanisme de blocage après 3 tentatives échouées

Au-delà de la simple vérification du mot de passe, un mécanisme de protection contre les tentatives de connexion répétées a été mis en place.

Chaque échec de connexion incrémente un compteur `nbTentatives` associé au compte concerné. Après trois échecs consécutifs, le compte est automatiquement marqué comme bloqué (`bloque = true`), empêchant toute connexion jusqu'à réactivation par un administrateur. Le compteur est réinitialisé à zéro après une connexion réussie.

4.3.2 Protection des routes

Chaque méthode de contrôleur réservée à un administrateur commence par un appel à la méthode utilitaire `isNotConnected(session)`.

Cette méthode vérifie la présence de l'attribut `admin` dans la session. En son absence, l'utilisateur est redirigé vers la page de connexion sans exécution du traitement.

4.4 Gestion des utilisateurs

Les fonctionnalités de gestion des utilisateurs offrent à l'administrateur :

— une vue paginée et triable de tous les comptes,

— une recherche par nom, prénom ou email,

— des actions rapides (éditer, désactiver, réactiver, supprimer).

Une attention particulière a été portée à la cohérence de l'interaction : lorsqu'un administrateur se supprime lui-même, sa session est immédiatement invalidée afin d'éviter toute incohérence entre l'état en base et la session active.

Chapitre 5

Interface Utilisateur (React + REST)

5.1 Principe général : deux applications indépendantes

Contrairement à l'interface d'administration où une seule application Spring Boot gère le traitement complet d'une requête jusqu'au rendu HTML, l'interface utilisateur repose sur une séparation stricte entre deux applications indépendantes.

L'application React, développée avec Vite, s'exécute côté client sur le port 5173. L'application Spring Boot expose quant à elle une API REST sur le port 8088. Le navigateur charge d'abord l'ensemble du code JavaScript de l'application React, puis effectue des appels HTTP vers le serveur afin de récupérer les données nécessaires au format JSON. Ainsi, aucune donnée n'est disponible initialement dans le client : l'état de l'application est construit progressivement à partir des réponses de l'API.

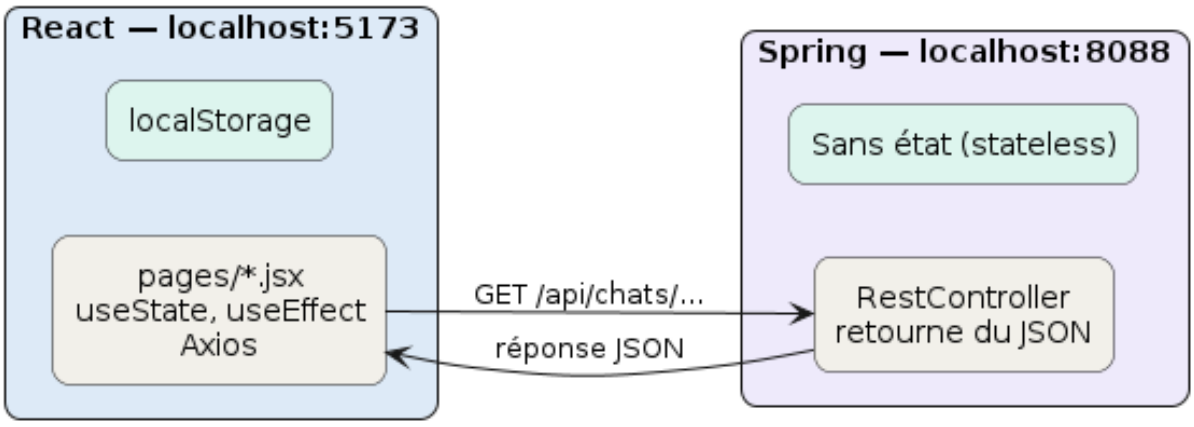


FIGURE 5.1 – Séparation entre React et Spring : communication par JSON sans session partagée

5.2 Absence d'état partagé entre React et Spring

Dans une architecture MVC classique, Spring conserve l'identité de l'utilisateur via `HttpSession`. Dans une architecture REST, ce mécanisme disparaît : chaque requête est indépendante et sans état (*stateless*).

Le serveur ne conserve aucune information sur les requêtes précédentes, et ne connaît l'utilisateur que par les données explicitement fournies dans chaque requête.

Pour pallier cette absence d'état côté serveur, l'application React stocke l'identité de l'utilisateur dans le `localStorage` :

```
localStorage.setItem("user", JSON.stringify(response.data));
```

Lors des appels API, cette information est récupérée et transmise explicitement :

```
const connectedUser = JSON.parse(localStorage.getItem("user"));
getOwnerChats(connectedUser.id); // GET /api/chats/owner/3
```

Le serveur reçoit donc un identifiant sans pouvoir vérifier directement son authenticité. Cette limite est assumée dans le cadre du projet et discutée dans la partie sécurité.

5.3 Organisation des services REST côté Spring

Trois contrôleurs annotés `@RestController` exposent les fonctionnalités nécessaires à l'application React. Contrairement aux contrôleurs MVC, ils ne retournent jamais de vue, mais des objets Java sérialisés automatiquement en JSON.

- **AuthRestController** : gestion de l'authentification et retour des données utilisateur.
- **UserRestController** : récupération des utilisateurs et des informations de base.
- **ChatRestController** : gestion complète des salons de discussion (création, suppression, invitations, récupération).

Un choix important d'architecture consiste à ne jamais exposer directement les entités JPA (`User`, `Chat`). Celles-ci sont transformées en DTO (Data Transfer Objects) afin d'éviter :

- la fuite de données sensibles (ex : mot de passe),
- les boucles infinies liées aux relations bidirectionnelles JPA.

Les DTO permettent de n'exposer que les champs nécessaires à l'affichage côté client.

5.4 Organisation du code côté React

L'application React est structurée en trois répertoires principaux :

— **services** : centralise tous les appels API via Axios.

— **pages** : composants représentant des vues complètes de l'application.

— **components** : composants réutilisables sans logique métier forte.

Le composant `ProtectedRoute` assure une protection côté client :

```
function ProtectedRoute({ children }) {
    const connectedUser = JSON.parse(localStorage.getItem("user"));
    if (!connectedUser) {
        return <Navigate to="/login" replace />;
    }
    return children;
}
```

Ce mécanisme améliore l'expérience utilisateur mais ne constitue pas une sécurité réelle, car il peut être contourné via les outils du navigateur.

5.4.1 Cycle de vie d'une page React

Le fonctionnement d'un composant React repose sur deux hooks principaux :

— `useState` : gestion de l'état local,

— `useEffect` : exécution de code lors du cycle de vie.

Exemple de déroulement :

```
Navigation vers "Mes salons"
→ useEffect déclenché
→ axios.get /api/chats/owner/{id}
→ ChatRestController.getOwnerChats()
→ ChatRepository.findByProprietaire()
→ réponse JSON (ChatDTO)
→ setOwnerChats(response.data)
→ re-render React
```

Contrairement à MVC (Thymeleaf), React met à jour uniquement une partie de l'interface sans rechargement complet de la page.

5.5 Fonctionnalités développées

L'application permet :

— la création de chats avec sélection multiple d'utilisateurs,
— la modification et suppression de chats,
— l'ajout d'invités à un chat existant,
— l'affichage des chats où l'utilisateur est propriétaire ou invité,
— une distinction visuelle entre les rôles des utilisateurs.

Chapitre 6

Service de chat WebSocket

6.1 Pourquoi WebSocket plutôt que HTTP classique

Le protocole HTTP, utilisé dans les architectures MVC et REST, repose sur un modèle requête-réponse : le serveur ne peut envoyer une information qu'en réponse à une requête du client.

Ce fonctionnement est incompatible avec un chat en temps réel, où le serveur doit pouvoir pousser un message dès sa réception, sans sollicitation préalable du client.

Le protocole WebSocket répond à cette limitation en établissant une connexion persistante et bidirectionnelle entre client et serveur.

6.2 Le choix de STOMP

Plutôt que d'utiliser l'API bas niveau `javax.websocket`, le projet s'appuie sur STOMP (Simple Text Oriented Messaging Protocol), qui se superpose à WebSocket.

STOMP introduit une notion de **topics** (canaux logiques), facilitant la gestion des abonnements sans avoir à maintenir manuellement les connexions.

Côté client, la bibliothèque SockJS est utilisée afin de fournir une solution de repli en cas d'indisponibilité de WebSocket.

6.3 Fonctionnement général

Schéma du flux de message STOMP Publish vers `/app` → traitement serveur → redistribution vers `/topic/chat/{id}` → réception par tous les clients abonnés.

La configuration définit :

— un endpoint `/ws`,

— un préfixe `/app` pour les messages envoyés au serveur,

— un préfixe `/topic` pour les messages diffusés aux clients.

Le contrôleur `ChatWebSocketController` reçoit les messages via `@MessageMapping` et les redistribue à l'aide de `SimpMessagingTemplate` vers des destinations dynamiques :

```
/topic/chat/{chatId}
```

Cette construction garantit l'isolation complète des salons de discussion.

6.3.1 Non-persistance des messages

Conformément au cahier des charges, aucun message n'est stocké en base de données.

La classe `ChatMessage` n'est pas une entité JPA : elle sert uniquement de structure de transport entre client et serveur.

Conséquence directe : un utilisateur rejoignant un chat en cours ne peut pas voir les messages envoyés avant son arrivée.

6.3.2 Liste des utilisateurs connectés et limite connue

Le serveur maintient en mémoire, pour chaque chat, la liste des utilisateurs connectés :

```
Map<Long, List<String>>
```

Cette liste est mise à jour lors des événements JOIN et LEAVE, puis diffusée aux clients abonnés.

Cette approche garantit une vision en temps réel des participants connectés.

Cependant, elle repose sur l'envoi explicite d'un message LEAVE. En cas de fermeture brutale du navigateur ou de perte réseau, l'utilisateur peut rester affiché comme connecté.

Une amélioration possible serait d'exploiter les événements Spring `SessionDisconnectEvent` pour détecter automatiquement les déconnexions.

Chapitre 7

Limites et pistes d'améliorations

7.1 Déploiement de l'application React non intégré à Spring

Une limite architecturale importante concerne le mode de déploiement de l'application React.

Le sujet du TD7 impose normalement un build de production via :

```
npm run build
```

afin de générer une application statique intégrée au serveur Spring Boot.

Dans le projet actuel, cette étape n'a pas été réalisée : l'application React est exécutée en mode développement via Vite sur le port 5173, indépendamment du serveur Spring Boot (port 8088).

Les deux applications doivent donc être lancées séparément, ce qui ne correspond pas à un déploiement de production et complexifie l'exécution du projet.

Une amélioration consisterait à :

— générer le build React ;

— placer les fichiers dans `src/main/resources/static` ;

— servir l'application directement via Spring Boot.

Cela permettrait d'unifier le déploiement sur un seul serveur et un seul port.

7.2 Autres améliorations fonctionnelles envisageables

Plusieurs améliorations fonctionnelles ont été identifiées au cours du développement.

7.2.1 Gestion des déconnexions WebSocket

La gestion des utilisateurs connectés repose actuellement sur l'envoi explicite d'un message `LEAVE` lors de la fermeture de la fenêtre de discussion. Cette approche ne permet pas de détecter correctement certaines déconnexions brutales, comme la fermeture du navigateur ou une perte de connexion réseau.

Une amélioration consisterait à exploiter les événements `SessionDisconnectEvent` fournis par Spring afin de détecter automatiquement ces situations et de maintenir une liste fiable des utilisateurs connectés.

7.2.2 Envoi réel des identifiants par courrier électronique

Conformément au cahier des charges, les nouveaux utilisateurs devraient recevoir leur mot de passe temporaire par courrier électronique.

Dans la version actuelle du projet, cette fonctionnalité est simulée par un affichage dans la console du serveur. L'intégration d'un service d'envoi d'emails, par exemple via Spring Mail, permettrait de se rapprocher davantage d'un fonctionnement réel.

Chapitre 8

Aspects complémentaires

8.1 Organisation du travail en équipe

Le projet a été réalisé en binôme, avec Git comme outil de collaboration et GitLab comme plateforme d'hébergement, Eclipse étant utilisé comme environnement de développement.

Chaque membre du binôme a travaillé sur sa propre branche, ce qui a permis d'avancer en parallèle sur des parties distinctes du projet sans bloquer l'autre. Des points réguliers ont permis de confronter les avancées de chacun, de discuter des choix effectués et de fusionner les branches une fois les fonctionnalités stabilisées.

Cette organisation visait autant la répartition du travail que l'apprentissage mutuel, chaque membre prenant connaissance du code de l'autre plutôt que de rester cantonné à sa seule partie.

8.2 Utilisation de l'IA

Un assistant d'intelligence artificielle a été utilisé en complément du développement, principalement pour deux usages.

Côté frontend, il a aidé à la production de code JavaScript, HTML et CSS répétitif : mise en forme et cohérence visuelle des pages d'administration, fonctions JavaScript (validation de formulaire, génération de mot de passe, filtrage dynamique) ainsi que la structuration de composants React.

Côté WebSocket, il a également contribué à la compréhension de certains mécanismes de configuration du serveur de chat temps réel, notamment la distinction entre les préfixes `/app` (messages envoyés par un client vers le serveur) et `/topic` (messages diffusés par le serveur vers les clients abonnés), ainsi que la construction dynamique des destinations associées à chaque salon de discussion.

L'intelligence artificielle a donc été utilisée comme outil d'assistance et d'explication technique, sans se substituer au travail de conception, d'intégration et de validation réalisé dans le cadre du projet.

Chapitre 9

Conclusion

Le projet **ChattyChat** a permis de mettre en œuvre, au sein d'une application unique et cohérente, trois architectures de communication fondamentalement différentes : une interface d'administration en MVC avec rendu HTML côté serveur, une interface utilisateur en React consommant une API REST sans état partagé, et un serveur de chat temps réel reposant sur WebSocket et le protocole STOMP.

L'ensemble des fonctionnalités attendues par le cahier des charges a été implémenté : gestion complète des utilisateurs et des salons côté administration, planification de chats avec sélection des invités côté utilisateur, et diffusion de messages en temps réel sans aucune persistance, conformément à l'exigence centrale du sujet.

La principale difficulté rencontrée a résidé dans l'articulation cohérente de ces trois paradigmes au sein d'un même projet, chacun reposant sur une gestion différente de l'état et de l'identité de l'utilisateur connecté : session HTTP côté MVC, stockage dans le `localStorage` côté React, et état conservé en mémoire côté WebSocket.

Cette difficulté a également constitué le principal apprentissage du projet. Elle a permis de comprendre que le choix d'une architecture n'est jamais neutre et qu'il impose des contraintes spécifiques concernant la circulation de l'information, la gestion de l'état et les mécanismes de sécurité.

Le projet présente néanmoins certaines limites, documentées tout au long de ce rapport : absence de mécanisme d'authentification robuste pour les routes REST, protection des routes React assurée uniquement côté client, déploiement de l'application React non intégré à Spring Boot, ou encore gestion incomplète des déconnexions WebSocket anormales.

Ces limites ne remettent pas en cause les objectifs atteints, mais constituent au contraire des pistes d'amélioration clairement identifiées dans une perspective d'évolution ou de mise en production.

Au-delà des aspects techniques, ce projet a permis de mettre en pratique de manière concrète l'ensemble des notions abordées au cours du module SR03 : modélisation des

données, persistance avec JPA, architecture MVC, développement d'API REST, programmation côté client avec React et communication temps réel via WebSocket.

Il a également permis de développer une compréhension plus fine des compromis inhérents aux choix architecturaux réalisés dans le développement d'applications web modernes.